CECS 327 Distributed Lodging — Milestone 2: Communication Mechanisms Report

Course: CECS 327, Section 02, CSULB **Repository:** https://github.com/haileyc1201/CECS327-Distributed-Lodging **Branches:** `main` (Python simulation) · `tom-react-frontend` (live Node + React + Cloudflare Workers/D1 version)

1. Overview: one protocol, two paths

The system has two implementations that use the **same message envelope and wire formats**:

- **Path A: Python simulation (`main`).** Each node is its own OS process: Property RPC, Reservation RPC, a pub-sub Broker, a Group Coordinator, 3 subscriber services and 3 property replicas. They talk over **raw TCP sockets** with length-prefixed frames carrying **JSON-RPC 2.0** and event messages. It builds on Hailey's Milestone 1 Property/Reservation socket services and the same `data/properties.json`. Her original services and client still run unchanged.
- **Path B: live web version (`tom-react-frontend`).** There are two runtimes:
- **Local Node stack.** Express microservices (Property 5001, Reservation 5002, Payment 5003, Gateway 8000), plus a new **Comm Hub** (5010) and 6 comm nodes (3 subscribers, 3 replicas) as separate processes, communicating with **HTTP JSON-RPC** and HTTP push.
- **Hosted Cloudflare Worker.** A Hono Worker with D1 that exposes **HTTP JSON-RPC** at `/api/rpc`. A **Durable Object (`CommHub`)** acts as broker and group coordinator. External subscribers and members connect over **WebSockets**. Every message is logged to a D1 table served at `/api/comm/logs`.

Because the envelope and the XML mapping are identical, a message marshalled by the Python codec can be unmarshalled by the Node/TypeScript codec. This was checked during development.

2. Communication model

Aspect	Path A: Python simulation (`main`)	Path B: Node local / Cloudflare hosted (`tom-react-frontend`)
Transport	TCP sockets (`socket` module), 4-byte length + 1-byte format header per frame	HTTP/1.1 (Node `fetch` + Express). Cloudflare: HTTPS to Worker, Worker→Durable Object stub fetch, WebSockets for push
Processes / nodes	10 OS processes: property-rpc :6001, reservation-rpc :6002, broker :6010, group-coordinator :6020, 3 subscribers, 3 replicas :6101-6103	Local: 11 processes (4 original services + hub + 3 subscribers + 3 replicas). Hosted: Worker isolates (stateless, global edge) + one `CommHub` Durable Object (single, strongly consistent)
Synchronous style	Blocking request-reply on a persistent socket	HTTP request-reply. The Worker awaits the DO reply
Asynchronous style	Broker pushes `event` frames over long-lived subscriber sockets	Local: hub POSTs `event` to each subscriber's `/deliver`. Hosted: DO pushes over WebSocket, built-in subscribers run inside the DO, mailbox members poll `group.inbox`
Shared state	In-memory per node; property data from `data/properties.json` (not modified by demo)	Local: CSV files in `data/`. Hosted: D1 (`properties`, `reservations`, `message_log`) + DO storage (membership, inboxes, sequence numbers)

Booking flow (both paths): **client → Reservation (RPC) → Property `availability.check` (request-reply) → mark unavailable → publish `booking.created` (pub-sub) → multicast `availability_update` to group `property-replicas` (group communication) → reply to client** listing which subscribers and replicas were reached.

3. Message formats

Every message, on every path, is an **envelope**:

```
{
  "id": "fc94cb9cacac",
  "type": "rpc.request",
  "sender": "client",
  "receiver": null,
  "timestamp": "2026-10-12T23:16:20.185Z",
  "correlation_id": null,
  "payload": {"jsonrpc": "2.0", "method": "reservation.book",
              "params": {"property_id": 101, "guest_name": "Demo Guest", "nights": 2}, "id": 3}
}
```

The reply carries `correlation_id` = the request's `id`, which lets clients match replies to requests:

```
{"id":"a13de2f6078d","type":"rpc.reply","sender":"reservation-rpc","receiver":"client",
 "timestamp":"2026-10-12T23:16:20.195Z","correlation_id":"fc94cb9cacac",
 "payload":{"jsonrpc":"2.0","id":3,"result":{"status":"accepted",
   "reservation":{"reservation_id":"R-27d48c43","property_id":101,"amount":300,"status":"confirme
d"},
   "notified_subscribers":["notification-service","analytics-service","audit-service"],
   "replicas_updated":["replica-1","replica-2","replica-3"]}}}
```

Pub-sub event and group multicast payloads:

```
{"type":"event","sender":"broker","payload":{"topic":"booking.created","seq":1,
  "publisher":"reservation-rpc","event":{"reservation_id":"R-27d48c43","property_id":101,"amount":30
0}}}

{"type":"group.deliver","sender":"group-coordinator","receiver":"replica-2","payload":{
  "group":"property-replicas","seq":1,"origin":"reservation-rpc",
  "message":{"op":"availability_update","property_id":101,"available":false}}}
```

Message types: `rpc.request`/`rpc.reply`, `subscribe`/`subscribe.ack`, `publish`/`publish.ack`, `event`/`event.ack`, `group.join`/`group.leave`/`group.members`/`group.multicast` (+ `.ack`/`.reply`), `group.deliver`, `group.view`, `group.ack`, `group.heartbeat`, `group.inbox` (hosted).

Second format: typed XML. The same envelope can be sent as XML. Each element carries a type attribute (`t="int|float|bool|str|null|dict|list"`), so the round-trip is lossless:

```
<message><envelope t="dict"><entry key="id"><value t="str">9aee0f075a81</value></entry>
<entry key="type"><value t="str">rpc.request</value></entry> ...
<entry key="payload"><value t="dict"><entry key="params"><value t="dict">
<entry key="property_id"><value t="int">103</value></entry></value></entry></value></entry></envelop
e></message>
```

4. Marshalling and unmarshalling

Step	Path A: Python (`src/communication/codec.py`, `transport.py`)	Path B: Node (`services/shared/messaging.js`) / Worker (`cloudflare/src/comm/marshal.ts`)
Marshal	`marshal(envelope, "json"/"xml")` → UTF-8 bytes (`json.dumps` / ElementTree)	`marshal(envelope, fmt)` → string (`JSON.stringify` / typed-XML serializer)

Step	Path A: Python (`src/communication/codec.py`, `transport.py`)	Path B: Node (`services/shared/messaging.js`) / Worker (`cloudflare/src/comm/marshal.ts`)
Framing	`[len:4][fmt:'J'/'X'][body]`. Removes Milestone 1's 1024-byte `recv` limit and allows many messages per connection	HTTP body. Format given by `Content-Type: application/json` or `application/xml`. The reply uses the same format
Unmarshal	`recv_exact` header → body → `unmarshal(body, fmt)` → dict	`formatFromContentType()` → `unmarshal(text, fmt)` (JSON.parse / small XML parser)
Before/after logging	Each SEND log stores `message` (before) and `wire` (after). Each RECV log stores `wire` (before) and `message` (after)	Identical fields. Hosted: stored in D1 `message_log.message` / `.wire`

5. Why sockets / RPC

	Path A: TCP sockets + JSON-RPC 2.0	Path B: HTTP JSON-RPC 2.0 (+ WebSockets)
Reason	Milestone 1 already used TCP sockets. Sockets show the low-level mechanics the course covers (connect, framing, blocking request-reply). JSON-RPC adds a standard method/params/error contract instead of ad-hoc `action` strings	Cloudflare Workers cannot open listening TCP sockets. Inbound traffic is HTTP/WebSocket only, so RPC over HTTP is the natural fit. JSON-RPC keeps the same method names as Path A (`availability.check`, `reservation.book`, ...). WebSockets give server push for pub-sub and group delivery to browsers or external nodes
Errors	JSON-RPC codes: -32600 invalid request, -32601 method not found, -32602 bad params, 404 property not found	Same codes
Methods	`rpc.ping`, `property.list/get`, `availability.check`, `property.set_available`, `reservation.book/get/list`	Plus `pubsub.publish/subscribers`, `group.join/leave/members/multicast/heartbeat/inbox`, `logs.recent`

RMI was not chosen because it is Java-specific, and the team's stacks are Python and JavaScript/TypeScript.

6. Publish-subscribe design

	Path A: Python	Path B: Node local	Path B: Cloudflare hosted
Broker	`Broker` process :6010	Comm Hub :5010	`CommHub` Durable Object
Subscribe	`subscribe {topics}` on a long-lived socket	`POST /subscribe {topic,name,url}`	Built-in services + `GET /api/comm/ws?role=subscriber&topic=…`
Subscribers in demo	notification-, analytics-, audit-service (3 processes)	same 3 (separate processes)	same 3 built-in + 2 WebSocket clients
Publish	Reservation sends `publish` → broker fans out `event` (with seq) to each subscriber → `publish.ack {delivered_to, failed}`	Same over HTTP. Subscriber replies `event.ack`	Same; WebSocket push
Decoupling / failure	Publisher does not know the subscribers. A broken socket is removed	A subscriber whose push fails is dropped	A closed WebSocket is dropped by the runtime

Topic filtering is tested: a publish on `other.topic` reaches nobody.

7. Group communication design

	Path A: Python	Path B: Node local	Path B: Cloudflare hosted
Coordinator	`GroupCoordinator` :6020	Comm Hub `/group/*`	`CommHub` DO (single instance → one consistent view)

	Path A: Python	Path B: Node local	Path B: Cloudflare hosted
Members	replica-1..3, each with its own TCP listener	replica-1..3 HTTP processes	mailbox members (HTTP join + inbox) and WebSocket members
Membership ops	`group.join/leave/members`; every change → `group.view {view_id, members}` to all	same	same + `group.heartbeat` lease (10 min)
Multicast	Coordinator sends a numbered `group.deliver` to every member and waits for `group.ack`. Replies `{seq, delivered_to, failed}`	same	WebSocket send / mailbox enqueue
Failure detection	Unreachable member is removed and a new view announced	same	Closed socket or expired lease is removed and a new view announced
Use	Keep property availability caches consistent across replicas	same	same

The demos kill `replica-3` (Python, Node) or close the WebSocket replica (Cloudflare). The next multicast reports `failed: ["replica-3"]` and the membership view shrinks.

8. Logging

Every SEND and RECV on every node is logged with an ISO-8601 UTC timestamp (millisecond precision, `Z`):

- **Python:** `logs/messages.jsonl` (`COMM_LOG_FILE` overrides the path), plus readable stdout lines.
- **Node:** `logs/messages.jsonl`, served at `GET /api/comm/logs` (gateway) and `GET /logs` (hub).
- **Cloudflare:** D1 table `message_log`, served at `GET /api/comm/logs?limit=&node=`. It persists across isolates.

9. Log excerpts from real runs

All excerpts were copied from runs on 2026-10-12. The full logs are in `docs/logs/`.

9.1 Path A: Python simulation (`python -m src.communication.demo`)

```
=== 2+3+4. reservation.book -> property RPC, pub-sub, multicast =
2026-10-12T23:16:55.257Z [client] SEND  -> 127.0.0.1:6002 type=rpc.request id=8ef12977323e fmt=json
bytes=266
2026-10-12T23:16:55.266Z [client] RECV  <- 127.0.0.1:6002 type=rpc.reply id=2a0f976ce60b fmt=json by
tes=579
  booking 101: {
  "status": "accepted",
  "reservation": {
    "reservation_id": "R-fa7a220d",
    "property_id": 101,
    "property_name": "Long Beach Apartment",
    "guest_name": "Demo Guest",
    "nights": 2,
    "amount": 300,
    "status": "confirmed",
    "created_at": "2026-10-12T23:16:55.260Z"
...
=== 4b. failure: kill replica-3, multicast again ================
2026-10-12T23:16:55.571Z [client] SEND  -> 127.0.0.1:6020 type=group.multicast id=c308b7a43408 fmt=j
son bytes=270
2026-10-12T23:16:55.576Z [client] RECV  <- 127.0.0.1:6020 type=group.multicast.ack id=906842448018 f
mt=json bytes=282
  multicast ack: {'group': 'property-replicas', 'seq': 2, 'delivered_to': ['replica-1', 'replica-
```

```
2'], 'failed': ['replica-3']}
2026-10-12T23:16:55.576Z [client] SEND  -> 127.0.0.1:6020 type=group.members id=de9e1fae51be fmt=jso
n bytes=194
2026-10-12T23:16:55.577Z [client] RECV  <- 127.0.0.1:6020 type=group.members.reply id=5cb1ab2d7a2f f
mt=json bytes=246
  group members now: ['replica-1', 'replica-2']

=== summary from logs/messages.jsonl ============================
  164 log entries, 73 SEND, 73 RECV
  notification-service   ['send confirmation email to Demo Guest for Long Beach Apartment']
  analytics-service      ['record revenue $300 for property 101']
  audit-service          ['append audit record for R-fa7a220d']
  replica-1              ['replica state now {101: False}', 'replica state now {101: False, 103: Tru
e}']
  replica-2              ['replica state now {101: False}', 'replica state now {101: False, 103: Tru
e}']
```

One SEND entry (booking request) with before/after marshalling:

```
{
 "ts": "2026-10-12T23:16:55.257Z",
 "node": "client",
 "event": "SEND",
 "peer": "127.0.0.1:6002",
 "format": "json",
 "bytes": 266,
 "message": {
  "id": "8ef12977323e",
  "type": "rpc.request",
  "sender": "client",
  "receiver": null,
  "timestamp": "2026-10-12T23:16:55.257Z",
  "correlation_id": null,
  "payload": {
   "jsonrpc": "2.0",
   "method": "reservation.book",
   "params": {
    "property_id": 101,
    "guest_name": "Demo Guest",
    "nights": 2
   },
   "id": 4
  }
 },
 "wire": "{\"id\":\"8ef12977323e\",\"type\":\"rpc.request\",\"sender\":\"client\",\"receiver\":nul
l,\"timestamp\":\"2026-10-12T23:16:55.257Z\",\"correlation_id\":null,\"payload\":{\"jsonrpc\":\"2.0
\",\"method\":\"reservation.book\",\"params\":{\"property_id\":101,\"guest_name\":\"Demo Guest\",\"n
ights\":2},\"id\":4}}"
}
```

9.2 Path B: Node local stack (`npm run comm:demo`)

```
2026-10-12T23:24:19.201Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=aa2856c
aa7ad fmt=json bytes=267
2026-10-12T23:24:19.267Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=6de0db09e
1a6 fmt=json bytes=789
2026-10-12T23:24:19.267Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=b864494
c61c4 fmt=json bytes=241
```

```
2026-10-12T23:24:19.274Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=6d3fa2375
971 fmt=json bytes=267
2026-10-12T23:24:19.277Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=5d4d946
a77f5 fmt=json bytes=324
2026-10-12T23:24:19.286Z [client] RECV  <- http://127.0.0.1:8000/api/rpc type=rpc.reply id=ac97244c4
24e fmt=json bytes=309
2026-10-12T23:24:19.286Z [client] SEND  -> http://127.0.0.1:8000/api/rpc type=rpc.request id=ec3bf4e
31079 fmt=json bytes=248
```


```
{
 "ts": "2026-10-12T23:24:19.262Z",
 "node": "replica-1",
 "event": "RECV",
 "peer": "127.0.0.1:44818",
 "format": "json",
 "bytes": 334,
 "wire": "{\"id\":\"a6b1d7da05d9\",\"type\":\"group.deliver\",\"sender\":\"comm-hub\",\"receiver
\":\"replica-1\",\"timestamp\":\"2026-10-12T23:24:19.261Z\",\"correlation_id\":null,\"payload\":{\"g
roup\":\"property-replicas\",\"seq\":1,\"origin\":\"reservation-service\",\"message\":{\"op\":\"avai
lability_update\",\"property_id\":101,\"available\":false,\"reservation_id\":\"res-722b39ce\"}}}",
 "message": {
  "id": "a6b1d7da05d9",
  "type": "group.deliver",
  "sender": "comm-hub",
  "receiver": "replica-1",
  "timestamp": "2026-10-12T23:24:19.261Z",
  "correlation_id": null,
  "payload": {
   "group": "property-replicas",
   "seq": 1,
   "origin": "reservation-service",
   "message": {
    "op": "availability_update",
    "property_id": 101,
    "available": false,
    "reservation_id": "res-722b39ce"
   }
  }
 }
}
```


9.3 Path B: Cloudflare Worker (`wrangler dev` + `cloudflare/scripts/comm-demo.mjs`)

```
=== 3. booking -> pub-sub + multicast ============================
  status accepted res-3b8716ac
  notified_subscribers [
  'analytics-service',
  'audit-service',
  'notification-service',
  'ws-subscriber-1-1360',
  'ws-subscriber-2-1360'
]
  replicas_updated [ 'replica-a-1360', 'replica-b-1360', 'replica-ws-1360' ]
  [PASS] booking accepted
  [PASS] built-in + websocket subscribers notified
  [PASS] all 3 replicas got multicast
  [PASS] ws subscriber 1 received event
  [PASS] ws subscriber 2 received event
  [PASS] ws replica received group.deliver
  [PASS] mailbox replica inbox has the update
```

```
  [PASS] double booking rejected

=== 4. failure / leave: websocket replica disconnects ===========
  members now: [ 'replica-a-1360', 'replica-b-1360' ]
  [PASS] closed websocket member removed from view

=== 5. message log (GET /api/comm/logs) ========================
  212 recent entries, 102 from this run
  2026-10-12T23:24:15.352Z [worker-gateway] RECV 127.0.0.1 rpc.request
  2026-10-12T23:24:15.356Z [worker-gateway] SEND 127.0.0.1 rpc.reply
  2026-10-12T23:24:15.363Z [worker-gateway] RECV 127.0.0.1 rpc.request
  2026-10-12T23:24:15.365Z [worker-gateway] SEND 127.0.0.1 rpc.reply
  2026-10-12T23:24:15.375Z [worker-gateway] RECV 127.0.0.1 rpc.request
  2026-10-12T23:24:15.379Z [worker-gateway] SEND 127.0.0.1 rpc.reply
  2026-10-12T23:24:15.385Z [worker-gateway] RECV 127.0.0.1 rpc.request
  2026-10-12T23:24:15.389Z [worker-gateway] SEND 127.0.0.1 rpc.reply
  [PASS] every entry has ISO-8601 ts
  [PASS] log has SEND and RECV for this run

ALL CHECKS PASSED
```

`GET /api/comm/logs` entry (pub-sub fan-out inside the Durable Object):


```
{
 "ts": "2026-10-12T23:23:16.502Z",
 "node": "comm-hub",
 "event": "SEND",
 "peer": "notification-service",
 "format": "json",
 "bytes": 489,
 "note": null,
 "message": {
  "id": "d87f3922dadb",
  "type": "event",
  "sender": "comm-hub",
  "receiver": "notification-service",
  "timestamp": "2026-10-12T23:23:16.502Z",
  "correlation_id": null,
  "payload": {
   "topic": "booking.created",
   "seq": 1,
   "publisher": "worker-gateway",
   "event": {
    "reservation_id": "res-1e2bc68a",
    "property_id": 101,
    "property_name": "Long Beach Apartment",
    "guest_name": "Demo ce07",
    "amount": 150,
    "payment_id": "pay-b670474e",
    "refund_id": null,
    "status": "confirmed",
    "created_at": "2026-10-12T23:23:16.495Z",
    "cancelled_at": null
   }
  }
 },
 "wire": "{\"id\":\"d87f3922dadb\",\"type\":\"event\",\"sender\":\"comm-hub\",\"receiver\":\"notific
ation-service\",\"timestamp\":\"2026-10-12T23:23:16.502Z\",\"correlation_id\":null,\"payload\":{\"to
pic\":\"booking.created\",\"seq\":1,\"publisher\":\"worker-gateway\",\"event\":{\"reservation_id
\":\"res-1e2bc68a\",\"property_id\":101,\"property_name\":\"Long Beach Apartment\",\"guest_name
```

```
\":\"Demo ce07\",\"amount\":150,\"payment_id\":\"pay-b670474e\",\"refund_id\"\u2026"
}
```

Deployment status: the Worker passes every check under `wrangler dev`, but it is **not deployed**. `wrangler deploy` failed 6+ times on 2026-10-12 (wrangler 4.135, 4.149 and latest, clean build, with backoff). Cloudflare returned HTTP 500 / code 10013 on `assets-upload-session`, while read calls with the same token work. The live site still serves the Milestone 1 build. To deploy: `npm run cf:deploy` (Node 22+), then `BASE_URL=https://lodging.pitchblack.icu npm run cf:test`.

10. Test cases

#	Test case	Python (`python -m unittest discover -s tests -t . -v`)	Node (`npm test`)	Worker (`npm run cf:test`, BASE_URL)
TC1	JSON + XML marshal round-trip, ISO timestamp	pass	pass	XML reply checked
TC2	Request-reply availability.check / property.get, JSON and XML, error codes	pass	pass	pass
TC3	Booking → 3 subscribers notified + 3 replicas updated; double booking rejected	pass	pass	pass (3 built-in + 2 WS subscribers; 2 mailbox + 1 WS replica)
TC4	Group join/view/multicast; failure removes member; leave	pass	pass	pass
TC5	Pub-sub topic filtering	pass	pass	n/a
TC6	Every log entry ISO-8601; SEND entries have before/after	pass	pass	pass

Results: Python **Ran 10 tests — OK**; Node **7/7 pass**; Worker **all demo checks passed (local wrangler dev)**. The raw outputs are in `docs/logs/*-test-results.txt` and `docs/logs/worker-local-demo-output.txt`.

11. Contribution table

Member	Contribution (communication milestone)
Member 1	*TBD*
Member 2	*TBD*
Member 3	*TBD*
Member 4	*TBD*
Member 5	*TBD*
Member 6	*TBD*
Member 7	*TBD*

Fill in names and contributions before submitting.